# CSC1110/CSC1122: Natural Language Technologies/ Foundations of NLP

## Lecture 5: Sequence Modelling

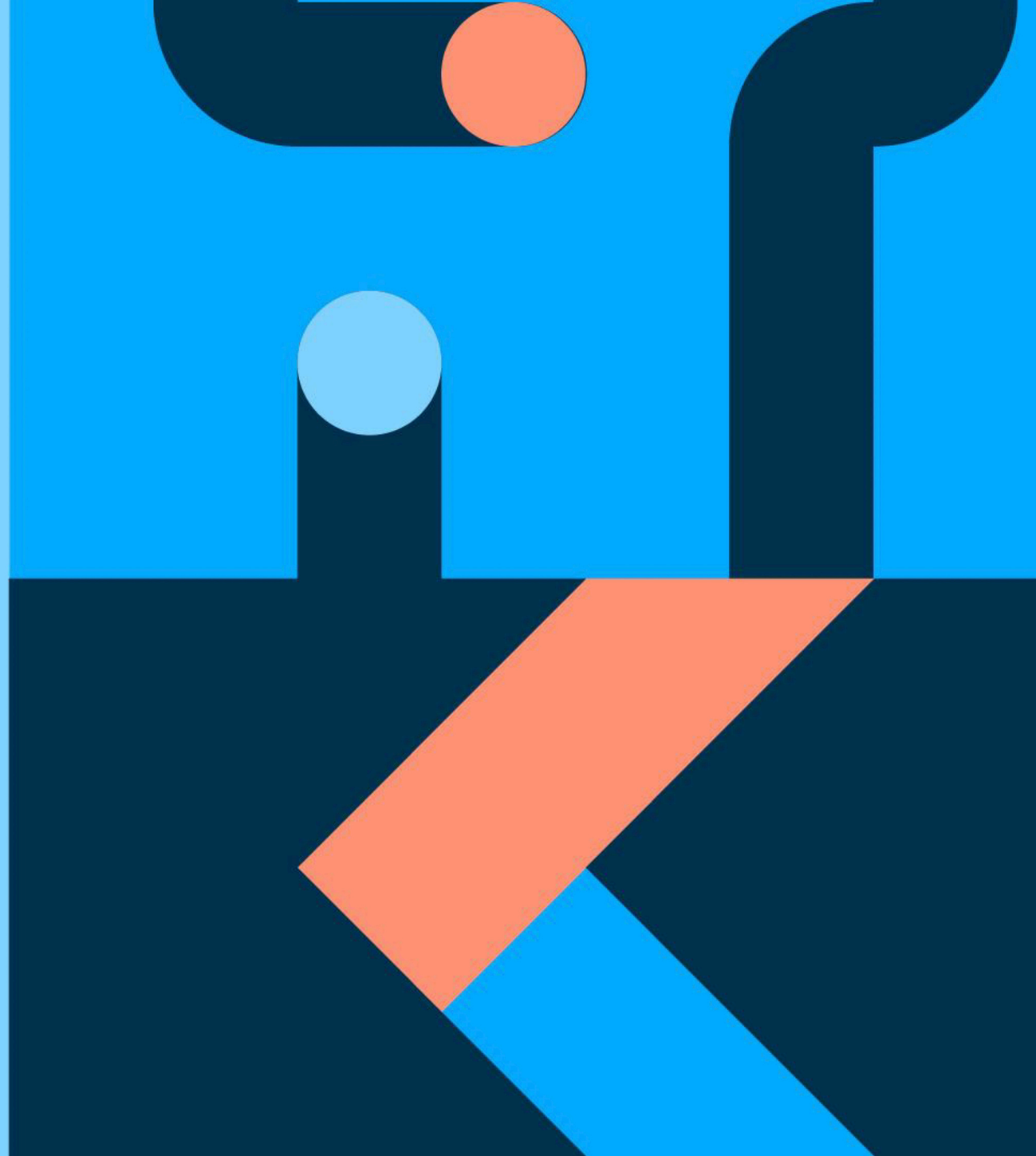
Dr. Ellen Rushe
Assistant Professor
School of Computing
Dublin City University

L2.24 (McNulty Building)
ellen.rushe@dcu.ie

DCU
Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

The contents of these slides are based on Chapter 8 of Jurafsky and Martin *Speech and Language Processing (3rd Edition)*

With special thanks to Prof. Jennifer Foster for a wealth of past slides!



# What is sequence labelling?

# What is Sequence Labelling?

*Input*: a sequence $X = x_1\ x_2 \ldots\ xn$

*Output*: a sequence $Y = y_1\ y_2 \ldots\ yn$

# Sequence Labelling Tasks in NLP

1. Part-of-speech Tagging
2. Named Entity Recognition
3. Language Identification

# Part-of-speech Tagging

## Input

*The plan was announced this evening in a live address by Leo Varadkar at Government Buildings.*

## Output

*The **det** plan **noun** was **verb** announced **verb** this **det** evening **noun** in **prep** a **det** live **adj** address **noun** by **prep** Leo **propn** Varadkar **propn** at **prep** Government **propn** Buildings **propn** . **punct***

# Named Entity Recognition

**Input**

*The plan was announced this evening in a live address by Leo Varadkar at Government Buildings.*

**Output**

*The plan was announced this evening in a live address by **[PERSON Leo Varadkar]** at **[LOCATION Government Buildings]** .*

**or**

*The **O** plan **O** was **O** announced **O** this evening **O** in **O** a **O** live **O** address **O** by **O** Leo **B-PER** Varadkar **I-PER** at **O** Government **B-LOC** Buildings **I-LOC** . **O***

# Language Identification

## Input

*Má tá AON Gaeilge agat, úsáid í! It's Irish Language Week .*

## Output

*Má **GA** tá **GA** AON **GA** Gaeilge **GA** agat **GA** , **PUNCT** úsáid **GA** í **GA** ! **PUNCT** It's **EN** Irish **EN** Language **EN** Week **EN** . **PUNCT***

Example *from https://aclanthology.org/W19-6905.pdf*

# Sequence Labelling Approaches

**Generative**
Hidden Markov Models

**Discriminative**
Conditional Random Fields

# More about part-of-speech tagging

# Why parts-of-speech are useful

Encode grammar of a language

Helpful in other NLP tasks

# Parts-of-speech in other NLP tasks

- Named entity recognition
- Syntactic parsing


- Sentiment analysis
  - Focus only on content words (nouns, verbs, adjectives, adverbs)

# Parts-of-speech in other tasks

POS N-grams
(NOUN, DET NOUN, ADJ ADJ NOUN)

Mixed part-of-speech and word n-grams
(not ADJ)

More abstract features
e.g.

- Number of adjectives per sentence/document
- Ratio of content to function words

# Tagsets

- Universal POS tagset: A part-of-speech tagset that can be applied to any language

| Open class words | Closed class words | Other |
|---|---|---|
| ADJ | ADP | PUNCT |
| ADV | AUX | SYM |
| INTJ | CCONJ | X |
| NOUN | DET | |
| PROPN | NUM | |
| VERB | PART | |
| | PRON | |
| | SCONJ | |

*https://universaldependencies.org/u/pos/all.html*

- Wall Street Journal tagset (much bigger and designed for English)

# English Part-of-Speech Tagging

An easy problem? A classifier that picks the most likely tag according to the training data gets 92% accuracy

A solved problem? English taggers achieve 97% accuracy

Not quite Taggers trained on newspaper text will struggle with more conversational language

*LOL PROPN ! PUNCT*

# Hidden Markov Models

# Hidden Markov Models

Find the most likely tag sequence for an input sequence of words

$$\hat{t}_1 \ldots \hat{t}_n = \text{argmax}\, p(t_1 \ldots . t_n | w_1 \ldots w_n)$$

# Hidden Markov Models

Find the most likely tag sequence for an input sequence of words

$$\hat{t}_1 \dots \hat{t}_n = \text{argmax}\, p(t_1 \dots t_n | w_1 \dots w_n)$$

$$p(t_1 \dots t_n | w_1 \dots w_n) = \frac{p(w_1 \dots w_n | t_1 \dots t_n) p(t_1 \dots t_n)}{p(w_1 \dots w_n)}$$

# Bayes Rule (again!)

$$p(t_1 \dots t_n | w_1 \dots w_n) = \frac{p(w_1 \dots w_n | t_1 \dots t_n) p(t_1 \dots t_n)}{p(w_1 \dots w_n)}$$

The denominator can be dropped because it is constant (we are choosing between different tag sequences for the same $w_1 \dots w_n$)

$$p(t_1 \dots t_n | w_1 \dots w_n) = p(w_1 \dots w_n | t_1 \dots t_n) p(t_1 \dots t_n)$$

# Simplifying assumptions

$$p(t_1 \dots t_n | w_1 \dots w_n) = p(w_1 \dots w_n | t_1 \dots t_n)\, p(t_1 \dots t_n)$$

Let's look at these two terms individually…..and make some simplifying assumptions

# Simplifying assumptions

$$p(t_1 \dots t_n | w_1 \dots w_n) = p(w_1 \dots w_n | t_1 \dots t_n)\, p(t_1 \dots t_n)$$

Let's look at these two terms individually.....and make some simplifying assumptions

$$p(w_1 \dots w_n | t_1 \dots t_n) \approx \prod_{i=1}^{n} p(w_i | t_i)$$

**Assume words occur independently from each other**

# Simplifying assumptions

$$p(t_1 \dots t_n | w_1 \dots w_n) = p(w_1 \dots w_n | t_1 \dots t_n)\, p(t_1 \dots t_n)$$

Let's look at these two terms individually.....and make some simplifying assumptions

$$p(w_1 \dots w_n | t_1 \dots t_n) \approx \prod_{i=1}^{n} p(w_i | t_i)$$

**Assume words occur independently from each other**

$$p(t_1 \dots t_n) \approx \prod_{i=1}^{n} p(t_i | t_{i-1})$$

**Assume current tag only dependent on previous tag**

# Two probabilities

$$p(t_1 \dots t_n | w_1 \dots w_n) \approx \prod_{i=1}^{n} p(w_i|t_i)\, p(t_i|t_{i-1})$$

Emission probability

Transition probability

# Two probabilities

$$p(t_1 \dots t_n | w_1 \dots w_n) \approx \prod_{i=1}^{n} p(w_i|t_i)\, p(t_i|t_{i-1})$$

Emission probability  Transition probability

**Where do we get the probabilities?**
Relative frequency estimation from a corpus that has been part-of-speech tagged – training corpus

# Estimating probabilities

## Emission probabilities

$$p(w|t) = \frac{\#\ word\ w\ tagged\ as\ t}{\#\ t}$$

## Transition probabilities

$$p(t_i|t_{i-1}) = \frac{\#t_{i-1}\ ti}{\#\ t_{i-1}}$$

# An Example: Part One

**Training Corpus**

*the **det** plan **noun** was **verb** announced **verb** this **det** evening **noun** in **prep** a **det** live **adj** address **noun** by **prep** Leo **propn** Varadkar **propn** at **prep** Government **propn** Buildings **propn** . **punct***

**Emission Probabilities**

p(the|det) = 0.333...
p(this|det) = 0.333...
p(a|det) = 0.333...

p(plan|noun) = 0.333...
p(evening|noun) = 0.333...
p(address|noun) = 0.333...

p(live|adj) = 1

p(was|verb) = 0.5
p(announced|verb) = 0.5

p(in|prep) = 0.333...
p(by|prep) = 0.333...
p(at|prep) = 0.333...

p(Leo|propn) = 0.25
p(Varadkar|propn) = 0.25
p(Government|propn) = 0.25
p(Buildings|propn) = 0.25

p(.|punct) =1

# An Example: Part Two

**Training Corpus**

*the **det** plan **noun** was **verb** announced **verb** this **det** evening **noun** in **prep** a **det** live **adj** address **noun** by **prep** Leo **propn** Varadkar **propn** at **prep** Government **propn** Buildings **propn** . **punct***

**Transition Probabilities**

p(verb|noun) = 0.333...
p(prep|noun) = 0.666...

p(verb|verb) = 0.5
p(det|verb) = 0.5

p(noun|det) = 0.666...
p(adj|det) = 0.333...

p(det|prep) = 0.333...
p(propn|prep) = 0.666...

p(noun|adj) = 1

p(propn|propn) = 0.5
p(prep|propn) = 0.25
p(punct|propn) = 0.25

# Visualising Transition Probabilities

p(det|det) = 0.05
p(noun|det) = 0.6
p(adj|det) = 0.35

p(det|noun) = 0.01
p(noun|noun) = 0.96
p(adj|noun) = 0.03

p(det|adj) = 0.03
p(noun|adj) = 0.67
p(adj|adj) = 0.3

# Implementation Detail

We have special transition probabilities for the start (and end) of a sequence so that we can estimate the probability of a particular part-of-speech tag starting (or ending) a sequence.

**For example**

Pstart(det) = 0.5 pend(det) = 0.0001
Pstart(noun) = 0.2 pend(noun) = 0.1
Pstart(verb) = 0.15 pend(verb) = 0.1
Pstart(punct) = 0.15 pend(punct) = 0.7999

# Finding the most likely sequence

Find the most likely tag sequence for an input sequence of words

$$\hat{t}_1 \ldots \hat{t}_n = \text{argmax} \prod_{i=1}^{n} p(w_i|t_i)\, p(t_i|t_{i-1})$$

The process of finding this sequence is known as decoding.

**Naïve approach**
For each possible tag sequence, find its probability
Rank the tag sequences by their probability and choose the one with the highest probability

# Number of possible sequences?

Given a 2-word sentence *Fish swim* and a tagset of 3, {noun,verb,adj}, how many different ways are there to tag the sentence?

1. noun noun
2. noun verb
3. noun adj
4. verb noun
5. verb verb
6. verb adj
7. adj noun
8. adj verb
9. adj adj

# Number of possible sequences?

Given a 3-word sentence *Fish sea swim* and a tagset of 2, {noun,verb}, how many different ways are there to tag the sentence?

1. noun noun noun
2. noun noun verb
3. noun verb noun
4. noun verb verb
5. verb noun noun
6. verb noun verb
7. verb verb noun
8. verb verb verb

# Number of possible sequences?

The number of possible sequences is  where *k* is the number of tags in the tagset and *n* is the length of the sequence.

**Exponential** in the length of the sequence

Infeasible to calculate the probability of every possible sequence

Instead, we use the **Viterbi** algorithm

# Viterbi algorithm

The Viterbi algorithm uses dynamic programming to efficiently compute the most likely sequence.

Dynamic programming solutions use a data structure to store solutions to a subproblem on the way to solving the full problem, thereby avoiding re-computation.

The probabilities of subsequences are computed only once and stored.

# Conditional Random Fields

# Disadvantage of HMMs

- HMMs tag a sequence looking only at the current word and the previous tag
- Make use of limited information when making a decision
- What other information might be useful?

# Other useful information

- Previous word ($w_{i-1}$), e.g. if $w_{i-1}$ is '*a*,' is likely to be a singular noun (some tagsets distinguish between singular and plural nouns)

# Other useful information

- Previous word ($w_{i-1}$) , e.g. if $w_{i-1}$ is '*a*,' is likely to be a singular noun (some tagsets distinguish between singular and plural nouns)
- Next word ( $w_{i+1}$ ), e.g. if $w_{i+1}$ is a past participle verb, e.g. *gone*, is likely to be a verb (*have/has/be/been, etc. gone*)

# Other useful information

- Previous word ($w_{i-1}$), e.g. if $w_{i-1}$ is '*a*,' is likely to be a singular noun (some tagsets distinguish between singular and plural nouns)
- Next word ($w_{i+1}$), e.g. if $w_{i+1}$ is a past participle verb, e.g. *gone*, is likely to be a verb (*have/has/be/been, etc. gone*)
- The shape of the word:
  - prefix, e.g. *un*acceptable
  - suffix, e.g. stroll*ed*
  - capitalized, e.g. *L*eo

# Conditional Random Fields

Conditional Random Fields are a **discriminative** approach to sequence labelling which can make use of all this information.

$$\hat{t}_1 \ldots \hat{t}_n = \text{argmax}\, p(t_1 \ldots . t_n | w_1 \ldots w_n)$$

Like logistic regression we directly estimate the labels given the input.

# Conditional Random Fields (CRFs)

Let's introduce X for the input sequence and Y for the output sequence.

$$p(t_1 \ \ldots \ t_n | w_1 \ \ldots w_n) \ = \ p(Y|X)$$

$$p(Y|X) = \frac{e^{\sum_{k=1}^{K} w_k . F_k(X,Y)}}{\sum_{Y' \in y} e^{\sum_{k=1}^{K} w_k . F_k(X,Y')}}$$

*F* is a feature which is applied to the whole sequence X.

*w* is the weight associated with the feature.

*K* is the total number of features.

y is the set of all possible sequences for X.

The denominator is the normalisation term that gives us a probability.

# Features in CRFs

$$F_k(X, Y) = \sum_{i=1}^{n} f_k(y_{i-1}, y_i, x_1 \ldots x_n, i)$$

$F_k$ is a global feature applied to the whole sequence
$f_k$ is a local feature applied to one word of the sequence
$f_k$ has access to

- Entire sequence of input words, $x_1 \ldots x_n$
- The current position, *i*
- The current and previous tag, $y_{i-1}$ (same as HMM) (linear-chain CRF)

# Feature Templates

Local features are defined using feature templates, e.g.

```
<yi,xi>,
<yi,xi-1>,
<yi,xi-1,xi+1>,
<yi,suffixxi>,
<yi, capitalizedxi>
```

These templates are instantiated from all the training data sequences to generate a large set of features.

# Features from Feature Templates

**Example**
Given the training sequence *the*/DET *dogs*/NOUN *bark*/VERB

The following features will be generated for $i = 2$

| Features | Value |
|---|---|
| $y_i$=NOUN, $x_i$=*dogs* | 1 |
| $y_i$=NOUN, $y_{i-1}$=*DET* | 1 |
| $y_i$=NOUN, $x_{i-1}$=*the*,$x_{i+1}$=*bark* | 1 |
| $y_i$=NOUN, $suffix_{x_i}$= *s* | 1 |
| $y_i$=NOUN, $capitalised_{x_i}$ | 0 |

Features are binary, i.e. have the value 1 or 0.

# Training and decoding

- As with LR, the weights associated with each feature are trained using stochastic gradient descent
- As with HMMs, decoding is done using the Viterbi algorithm

# That’s it for now!

DCU
Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University